

Detecting Market Manipulation in Limit Order Book Data

Kaggle ID: zcshlh, haotiantylerzhuo, haolinliunyu

1 Introduction

This project addresses the problem of detecting suspicious trading behavior from high-frequency limit order book (LOB) data. The dataset contains millisecond-level market events, including bid and ask returns, derivatives of returns, order sizes, trade indicators, cancellation indicators, and sequence numbers describing the order-event structure. The goal is to identify potentially manipulative events in the test set, such as spoofing, layering, quote stuffing, or other abnormal order book patterns.

This problem is challenging for two main reasons. First, the training and validation sets contain only normal samples, so the task is not a standard supervised classification problem. Second, the data are highly structured in time and are generated by multiple symbols across many trading sessions. Because of this, the model must learn what normal behavior looks like and then assign high anomaly scores to events that deviate from those patterns.

In this project, we framed the task as an anomaly detection problem. Instead of trying to predict labeled fraud classes directly, we trained a model on normal data only and used it to score the test set according to how unusual each event appears relative to the normal training distribution.

2 Methodology

Our main model is an Isolation Forest trained on engineered features derived from the raw LOB event stream. We chose this model because it is a strong baseline for unsupervised anomaly detection, scales reasonably well to large tabular datasets, and can capture unusual combinations of features without requiring positive fraud labels.

The workflow consisted of four main stages:

1. **Data loading and preprocessing.** The training, validation, and test CSV files were loaded with memory reduction to make the workflow practical on a large dataset. Numeric columns were downcast where possible to reduce RAM usage.
2. **Feature engineering.** We constructed features from both the raw numeric columns and several domain-motivated transformations:
 - Raw numeric inputs from the LOB dataset
 - Bid-ask spread, bid-ask ratio, and mid-return
 - Absolute return and derivative magnitude features
 - Size imbalance and total trade/cancel size features
 - Rolling statistics on `ReturnBid1` and `ReturnAsk1`
 - Event density features based on the number of events in recent millisecond windows

- Time gap features such as `time_delta` and inverse time delta
- Order structure features derived from `OriginalSequenceNumber`, including whether an event is a sub-event, sub-event depth, sub-event index, and total events within a base sequence
- Group-wise z-score features for important numeric columns

A key design decision in the final version was to compute rolling features, time-density features, and normalization features within each `Date + ExternalSymbol` group rather than only by symbol. This prevents events from different trading sessions from being mixed together and makes the temporal features more realistic.

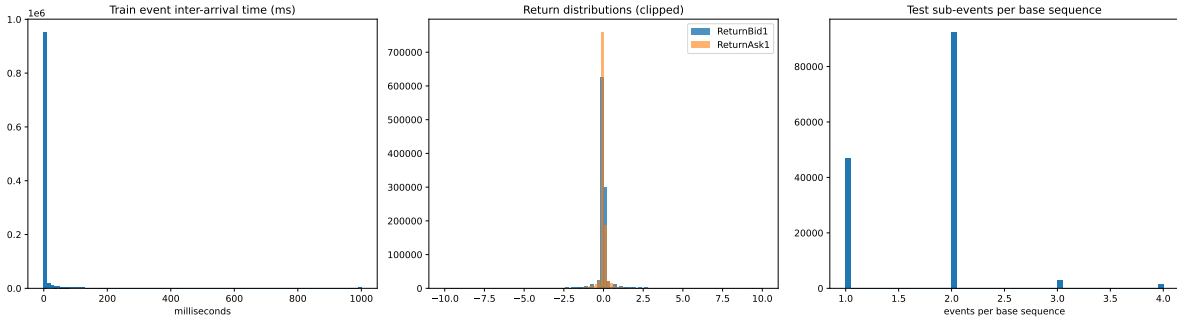


Figure 1: Exploratory distributions of inter-arrival time, returns, and order sequence structure.

Figure 1 shows several important properties of the LOB data that motivated our feature engineering. The inter-arrival time distribution is highly concentrated near zero, which supports the use of event-density and time-gap features. The return distributions are sharply centered but contain extreme tails, motivating absolute return and derivative magnitude features. The sub-event count distribution also shows repeated events within the same base sequence, which supports the construction of sequence-based features from `OriginalSequenceNumber`.

3. **Feature matrix construction and scaling.** After engineering the features, we aligned the train, validation, and test matrices so they shared the same columns. Missing and infinite values were replaced, and the final feature matrix was standardized using `StandardScaler`.
4. **Model training and scoring.** We trained an Isolation Forest with the following main settings:
 - `n_estimators = 300`
 - `max_features = 0.8`
 - `contamination = 0.01`
 - `random_state = 42`
 - `n_jobs = -1`

The model was fit on the normal training set only. We then used the model’s decision function to generate anomaly scores for validation and test data. These scores were converted into normalized anomaly scores in the range $[0, 1]$, where larger values indicate more suspicious behavior.

3 Analysis

The final feature matrix contained 76 engineered features. After training the Isolation Forest on the normal training data, we evaluated the score distribution on the validation set and used the validation scores to define a threshold consistent with the contamination setting.

The main run produced the following results:

- Training feature matrix: $1,070,022 \times 76$
- Validation feature matrix: $467,939 \times 76$
- Test feature matrix: $289,101 \times 76$
- Validation threshold at contamination 1%: 0.5273
- Validation rows flagged as anomalous: 4,680/467,939
- Mean validation anomaly score: 0.1412
- Mean test anomaly score: 0.2286
- Current Kaggle score: **0.97223**

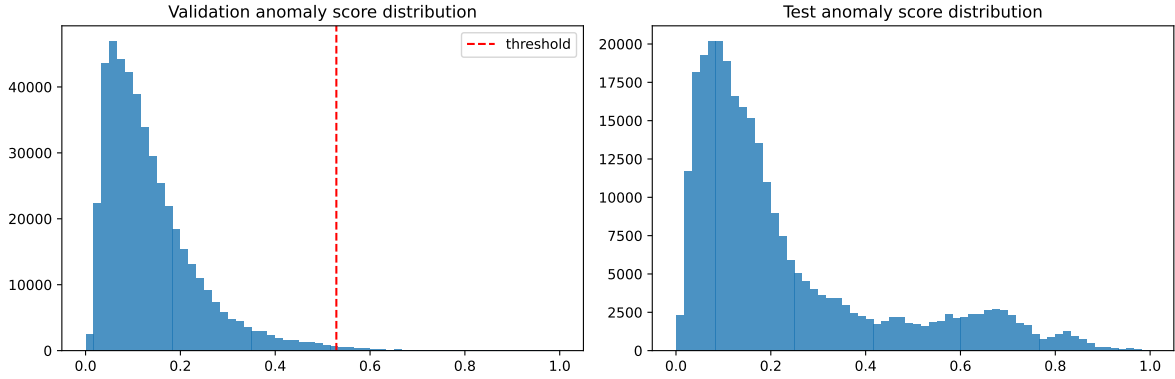


Figure 2: Validation and test anomaly score distributions. The dashed line indicates the validation threshold corresponding to 1% contamination.

Figure 2 provides evidence that the model assigns most validation observations relatively low anomaly scores, while a small right-tail region is treated as abnormal under the 1% contamination threshold. The test distribution has a heavier right tail than the validation distribution, suggesting that the test set contains more events that deviate from the normal training and validation patterns. This is consistent with the goal of the competition, where suspicious market manipulation events are expected to appear in the test set rather than in the normal training data.

The Kaggle score of **0.97223** suggests that the final anomaly scores are capturing a meaningful portion of the hidden manipulation patterns in the test set. This result is important because the model was trained without positive fraud labels. In other words, the score was achieved mainly through feature design and careful handling of temporal structure rather than through direct supervised fitting to fraudulent examples.

The most important improvement in this project came from changing the feature-engineering logic so that temporal and rolling features were computed within each trading session and symbol, rather than across all rows of the same symbol. Earlier logic grouped primarily by `ExternalSymbol`, which could unintentionally connect events from different dates. That made the rolling windows and timing-based features less reliable because the model was effectively comparing behavior across separate sessions as if they were part of one continuous sequence.

In the final version, grouping by `Date + ExternalSymbol` improved the validity of the engineered features in several ways:

- It made `time_delta` and event-density features session-aware, so event bursts were measured within a realistic local time context.
- It made rolling statistics reflect local market behavior instead of blending unrelated historical segments.
- It made sequence-level counts such as `base_seq_total_events` more meaningful by counting events within a session rather than across the entire dataset.

The top anomalies identified by the model were concentrated mainly in MSFT and INTC, and many of them showed extremely large `ReturnBid1` values combined with unusual derivative behavior and repeated sub-events in `OriginalSequenceNumber`. This is consistent with the kind of abnormal microstructure activity expected in market manipulation scenarios. In other words, the model did not simply flag random outliers; it surfaced events that appear extreme in both price-related and structural dimensions.

Examples from the highest-scoring anomalies include multiple MSFT and INTC events with anomaly scores close to 1.0, such as 22582680-011009-021 and 284753519-410692-017. These rows were characterized by very large bid-return values, abnormal derivative returns, and repeated sequence fragments, which is exactly the type of behavior the engineered features were designed to capture. This gave additional confidence that the model was learning project-specific manipulation signals rather than arbitrary noise.

We also considered other possible extensions, such as autoencoders or multi-model ensembles, but they were not included in the final submission because the project requirement specified that only the main model should be submitted. For that reason, we kept the final implementation focused on a single, interpretable anomaly-detection pipeline built around Isolation Forest.

One limitation of this project is that the validation set also contains only normal samples, so it is not possible to measure a standard supervised metric such as accuracy, F1 score, or AUC during development. Because of that, model development relied on the stability of the anomaly-score distribution, the plausibility of the top-ranked anomalies, and the quality of feature design. This makes feature engineering especially important, since improvements must come from representing abnormal behavior more clearly rather than from fitting to labeled fraud examples.

4 Conclusion

In this project, we approached LOB market manipulation detection as an anomaly detection problem and built a single-model solution based on Isolation Forest. The strongest part of the pipeline was the feature engineering, especially the use of session-aware grouping by `Date + ExternalSymbol` for rolling, timing, and sequence-based features. This change made the model better aligned with the structure of the trading data and reduced the risk of mixing unrelated events across trading sessions.

Overall, the final system successfully produced normalized anomaly scores for all test events and highlighted suspicious patterns characterized by extreme returns, abrupt derivative changes, dense event activity, and repeated sub-events. The project demonstrates that even without fraud labels, a carefully designed anomaly-detection workflow can extract meaningful signals from high-frequency LOB data. If we were to extend this work further, the next steps would be sequence-level aggregation, session-level calibration, and controlled comparison against additional unsupervised detectors.

The current Kaggle score of **0.97223** indicates that this final single-model pipeline is effective and competitive for the task. More importantly, the score supports the main conclusion of this project: in this competition, performance depends more on building session-aware, market-specific features than on using a more complicated model. The final Isolation Forest system therefore provides a practical and interpretable solution that matches both the structure of the dataset and the project submission requirements.